KODBTW / VIVA PREPARATION

Team lead · API and authentication

Sajal Goyal

This guide follows the repository's REST API and sign-in flow. Be ready to explain how a request is authenticated and how user data stays scoped to the signed-in account.

Contribution reminder: This is a repository-aligned role guide. Describe work you personally performed; confirm individual contribution details with your team.

Explain the system in one flow

React client	**THEN**	**REST endpoint**	**THEN**	**Spring Security + JWT filter**	**THEN**	**Controller / service**	**THEN**	**Repository → MySQL**

Core ideas to explain

Registration

Email is trimmed and lowercased, uniqueness is checked, and the password is encoded with BCrypt before the user is saved.

Login

The submitted password is checked with the password encoder. On success, the API returns a signed JWT and a safe user response.

JWT validation

The token subject identifies the user. The request filter validates the token and loads the user into Spring Security's request context.

Route policy

Registration and login are public. Health and public API reads are public; other API routes require authentication. Leaderboard sync is ADMIN-only.

Client session

The frontend sends a Bearer token. A 401 clears the stored token and signals the app to end the session; `/me` checks restored sessions.

Viva quick preparation

Use these as short answer outlines. Explain the idea in your own words and point to the relevant repository code.

Why hash passwords?

BCrypt is a one-way password hash with a per-password salt. The application verifies a login with matches(); it does not decrypt stored passwords.

What does stateless mean here?

The server does not keep an HTTP session for each user. Each protected request carries a signed Bearer token that the server validates.

How is a user identified from a token?

The JWT subject is the user ID. The filter validates the signature and uses that ID to load the user and establish the authenticated principal.

What happens when the frontend receives 401?

The API client removes the saved token and emits the unauthorized event so the app can return to an unauthenticated state.

What is the token timeout?

The configured default expiration is 86,400,000 ms, or 24 hours. The signing key and runtime settings come from configuration; never disclose secret values.

Where to point in the repository

```
01 backend/src/main/java/com/kodbtw/controller/AuthController.java
02 backend/src/main/java/com/kodbtw/service/AuthService.java
03 backend/src/main/java/com/kodbtw/service/UserService.java
04 backend/src/main/java/com/kodbtw/security/JwtService.java
05 backend/src/main/java/com/kodbtw/security/JwtAuthenticationFilter.java
06 backend/src/main/java/com/kodbtw/config/SecurityConfig.java
07 frontend/src/api/apiClient.ts
```

30-second presentation

“I’ll explain the API and authentication path. A user registers with a normalized email and BCrypt-protected password, then logs in to receive a signed JWT. The client sends that token as a Bearer credential. Spring Security validates it on protected requests and associates the request with the user. Route rules keep authentication endpoints public and protect the rest. The frontend also handles expired or rejected sessions by clearing the token.”

Be precise Do not call JWT encryption: it is signed, not a place to store secrets. Do not claim passwords are recoverable or that a 401 always means the token expired; it can mean any authentication failure.